SECURITY AUDIT FINAL PUBLIC REPORT

PAKTOL

RESUME

Audit report of the security audit conducted for Paktol Organization

Crypto4All
hello@crypto4all.com
26/08/2026

TABLE OF CONTENTS

1. AUDIT CONTEXT 2
2. REVISION HISTORY & VERSION CONTROL 3
3. FINDINGS 4
A. Summary 4
B. Overall Findings Remediation State 4
C. Overall Findings Table 5
D. Detailed Findings 6
F-04 — `convertToAssets` vs real liquidity dependency 6
F-02 — Premium points-gating absent (README vs code) 7
F-03 — Direct donations counted as yield 8
F-01 — Cooldown griefing via `_update` 9
F-05 — Treasury feeShares dilution 10
F-06 — `depositUpToCap()` reverts instead of returning `(0,0)` 10
F-07 — Missing `rescueToken()` 11
F-08 — Incomplete resumption post-emergency exit 12
E. Deployment Decision 12
4. AUDITING METHODOLOGIES 13
A. Manual Review 13
B. Automated Review 13
C. Analysis and Tools 14
5. DISCLAIMER 14

1. AUDIT CONTEXT

The Ethereum Virtual Machine blockchain enables decentralized, trustless, transparent, traceable, and secure execution of smart contracts. These characteristics are guaranteed only if the implementation adheres to industry best practices. In this context, smart contracts serve as the embodiment of logic and trust between parties. In this report, we have analyzed the code's effectiveness and security to provide technical recommendations and suggested improvements.

The purpose of this audit is to identify flaws and vulnerabilities that may have gone unnoticed during the development phase. Various types of issues can be detected: some may impact the functionality of the smart contract application, while others may result in insufficient protection in certain areas.

To this end, we conducted a thorough source code analysis to pinpoint areas that require patches. Our audit was comprehensive to ensure the identification of as many flaws and vulnerabilities as possible. It is important to note that this audit does not provide any guarantees regarding the utility, safety, suitability of the business model, compliance with regulatory frameworks, or other aspects related to the code.

Audit scope

> **Date:** 2026-08-26
> **Contract analyzed:** `PaktolVaultV2.sol - commit 4348b79`
> **Target network:** Ethereum (Byzantine Finance VaultV2 / EURC)
> **Scoring framework:** DeFi Security Scoring Framework v2.1
> **Applied domain profile:** Profile B — Yield Vault
>
> **Methodology:** In-depth manual review + automated review (static analysis + symbolic analysis + fuzzing)
>
> **INITIAL OVERALL SCORE BEFORE REMEDIATIONS: 77.3 / 100**
> **FINAL OVERALL SCORE AFTER REMEDIATIONS: 100 / 100**

This report was prepared to identify security findings in the smart contract code and ensure its robustness against potential threats. The audit included an extensive review process using static analysis, structural analysis, and manual line-by-line inspection, as described in the "Auditing Approach and Methodologies" section. The key focus areas of the audit were:

1. **Attack Resistance:** Testing the contract against common and uncommon attack scenarios to identify vulnerabilities.
2. **Best Practices Compliance:** Ensuring the code adheres to current industry standards and follows established best practices.
3. **Code Review:** Conducting a detailed manual review of every line of code to detect potential issues.

The security assessment revealed findings ranging from **medium to informational**. To enhance the contract's security and ensure compliance with industry standards, we recommend implementing the following improvements:

- **Robust Testing:** Test the contract against all potential attack vectors to enhance security.

- **Code Optimization:** Improve the structure and organization of the code for maintainability and clarity.
- **Comprehensive Unit Testing:** Add thorough unit tests to cover all possible use cases and scenarios.
- **Enhanced Documentation:** Include detailed comments for each function to improve code readability, especially for publicly verified contracts.
- **Transparent Operations:** Clearly outline and document privileged actions and processes once the protocol is live to ensure transparency.

By addressing these findings, the smart contract can achieve a higher level of security and align with industry's best practices, ultimately building trust among users and stakeholders.

2. REVISION HISTORY & VERSION CONTROL

Version	Date	Stakeholders	Description
0.1	02/07/2026	Kevin Hyund	Drafted the initial version of the audit intermediary report
0.2	06/07/2026	Christophe Ozcan	Reviewed and delivered the audit intermediary report.
1.0	07/07/2026	Philippe Lecornier (Paktol)	Validated the intermediary report delivery.
1.1	07/07/2026	Christophe Ozcan	Monitored remediation progress by opening issues on GitHub.
1.2	30/07/2026	Philippe Lecornier (Paktol)	Completed remediation of findings in the codebase and conducted testing scenarios.
1.3	24/08/2026	Christophe Ozcan	Second code reviewed and validated remediations.
2.0	26/08/2026	Christophe Ozcan	Conducted the final audit and prepared the findings resolution report.

The revision process of the audit report followed a structured approach, starting with an initial draft prepared by Kevin Hyund on July 2, 2026. Christophe Ozcan reviewed and delivered the intermediary report on July 6, 2026. By July 7, 2026, Philippe Lecornier (Paktol) validated the delivery of the intermediary report.

Subsequent updates focused on remediation progress. Christophe Ozcan monitored progress by opening GitHub issues on July 7, 2026. On July 30, 2026, Philippe Lecornier (Paktol) completed the remediation of findings in the codebase and conducted testing scenarios. Christophe Ozcan then performed a second code review and validated the remediations on August 24, 2026, before conducting the final audit and preparing the findings resolution report on August 26, 2026.

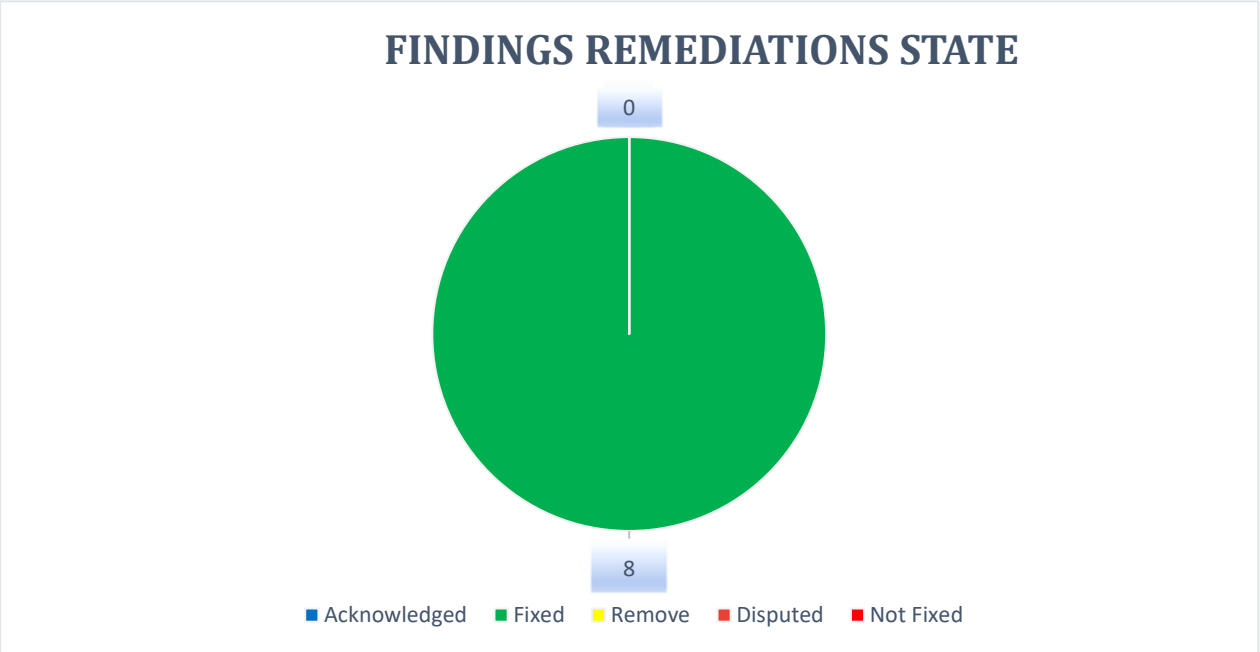
This timeline reflects a thorough and collaborative effort to ensure the report's accuracy and address all identified findings effectively.

3. FINDINGS

A. Summary

Overall, PaktolVaultV2.sol rests on solid technical foundations. Our initial assessment identified no vulnerabilities that would allow an unprivileged third party to directly steal users' principal.
However, the review highlighted key accounting regressions relative to the mature V1 baseline and an unmitigated structural liquidity risk regarding underlying redemptions from Byzantine Finance/Morpho. In accordance with our Smart Contract Security for DeFi scoring framework (annexed to this report), we identified 8 findings ranging in severity from Medium to Informational, categorized from P01 to P3 to effectively structure remediation priorities.
Following the initial review, the team fully remediated all identified findings, as detailed in the status section of this report. All architectural regressions and liquidity risks have been resolved, validating the contract for deployment and TVL growth.

B. Overall Findings Remediation State



Remediation State	Count	Findings	Definition
Fixed	8	F-04,F-02,F-03,F-05,F-01,F-06,F-08,F-07	Fix implemented and verified during retest.
Not Fixed	-		No effective fix applied, or fix failed verification.
Acknowledged	-		Risk accepted with governance sign-off.
Disputed	-		Validity or severity of finding is contested.

Remediation State	Count	Findings	Definition
☐ **Remove**	-		Finding invalidated through dispute resolution.

C. Overall Findings Table

ID	Finding	Score	Severity	Priority	State
F-04	`convertToAssets` vs real liquidity dependency	**48**	🟡 Medium	**P1**	🟢 Fixed
F-02	Premium points-gating absent (README vs code)	**40**	🟡 Medium	**P1**	🟢 Fixed
F-03	Direct donations counted as yield	**40**	🟡 Medium	**P2**	🟢 Fixed
F-05	Treasury feeShares dilution	**37.6**	🟢 Low	**P2**	🟢 Fixed
F-01	Cooldown griefing via `_update`	**15**	🟢 Low	**P1**	🟢 Fixed
F-06	`depositUpToCap()` reverts instead of returning `(0,0)`	**15**	🟢 Low	**P3**	🟢 Fixed
F-08	Incomplete resumption post-emergency exit	**10**	**i** Info	**P3**	🟢 Fixed
F-07	Missing `rescueToken()`	—	**i** Info	**P3**	🟢 Fixed

D. Detailed Findings

F-04 — `convertToAssets` vs real liquidity dependency

- **Severity:** 🟡 Medium — Score: **48** — Priority: **P1**
- **Category:** Liquidity risk / external protocol integration
- **Location:** `_byzantineLiquid()`, `maxWithdraw()`, `maxRedeem()`, `harvest()`
- **Confidence:** High
- **Source:** Internal audit

6

Description

`_byzantineLiquid()` values the Byzantine position via `convertToAssets` — an **accounting valuation**, not a guarantee of available liquidity:

```
    function _byzantineLiquid() internal view returns (uint256) {
        uint256 idle      = IERC20(asset()).balanceOf(address(this));
        uint256 byzShares = BYZANTINE_VAULT.balanceOf(address(this));
        uint256 byzAssets = byzShares > 0
            ? BYZANTINE_VAULT.convertToAssets(byzShares) : 0;
        return idle + byzAssets;
}
```

Yet `_withdraw` attempts a real withdrawal:

```
    if (idle < assets) {
        BYZANTINE_VAULT.withdraw(assets - idle, address(this),
address(this));
    }
```

Under Morpho liquidity stress (over-utilized markets, withdrawal cascade), `BYZANTINE_VAULT.maxWithdraw()` drops well below `convertToAssets()`. Result: `maxWithdraw` on the PaktolVault side **promises available funds**, but the actual `withdraw` call **reverts**. The code comment even admits bypassing Byzantine's `maxWithdraw()` — amplifying the divergence.

Mitigating Factors

The situation is self-resolving once Morpho liquidity is restored. Funds are not lost, only temporarily inaccessible.

Recommendation 1

Introduce a dedicated function for real liquidity, distinct from the accounting valuation.

Recommendation 2

Use `_byzantineWithdrawable()` in `maxWithdraw` / `maxRedeem`. Retain `convertToAssets` **exclusively** in `totalAssets()` (accounting value), never as a liquidity proxy.

Remediation

The issue is now resolved by the patched code version validated by the commit **5f885c3.**

F-02 — Premium points-gating absent (README vs code)

- **Severity:** 🟡 Medium — Score: **40** — Priority: **P1**
- **Category:** Product compliance / documentation-implementation divergence
- **Location:** `deposit()`, `mint()`, `depositWithPermit()`, `depositUpToCap()`
- **Confidence:** High
- **Source:** Internal audit + README
- **Scoring note:** The floor is triggered via **Integrity=High** (the README/code divergence is real), *not* via Yield=Critical. Granting 5% instead of 3.5% is a *business* impact, not a security catastrophe — hence Medium and not High.

Description

The README states that V2 Premium is **points-gated**. The contract exposes the associated variables and error:

```
mapping(address => uint256) public points;
uint256 public premiumThreshold;
error NotEnoughPoints(uint256 current, uint256 required);
```

But **no** deposit function checks `points[receiver] >= premiumThreshold`. Only `premiumExpiry` is verified:

```
if (REQUIRES_AUTH && block.timestamp > premiumExpiry[receiver])
    revert PremiumAccessExpired(premiumExpiry[receiver]);
```

`setPoints` and `premiumThreshold` are therefore **dead functional code** with respect to gating.

Recommendation

Two mutually exclusive options:

Option A — Implement on-chain gating:

```
    function _checkPremiumAccess(address user) internal view {
        if (!REQUIRES_AUTH) return;
        if (block.timestamp > premiumExpiry[user])
            revert PremiumAccessExpired(premiumExpiry[user]);
        if (points[user] < premiumThreshold)
            revert NotEnoughPoints(points[user], premiumThreshold);
    }
```

8

Called in `deposit`, `mint`, `depositWithPermit`, `depositUpToCap`.

Option B — Assume off-chain verification: remove/rename `points` and `premiumThreshold`, and correct the README (`"backend-authorized access"`).

Remediation

The issue is now resolved by the patched code version validated by the commit **86aba83. The option B was integrated.**

F-03 — Direct donations counted as yield

- **Severity:** 🟡 Medium — Score: **40** — Priority: **P2**
- **Category:** Accounting logic — regression vs V1
- **Location:** `totalAssets()`, `harvest()`
- **Confidence:** High
- **Source:** Internal audit
- **Scoring note:** Floor triggered via **Integrity=High** (the donation genuinely corrupts accounting), *not* via Yield. Same reasoning as F-02.

Description

`totalAssets()` includes the idle EURC balance without explicit accounting tracking:

```
    return bzyValue + IERC20(asset()).balanceOf(address(this)); // ← donations included
```

A direct EURC transfer inflates `totalAssets()`, and therefore `grossYield = current - lastTotalAssets` at the next harvest. **V1 had solved this problem via `_idleBalance` — V2 is a regression.**

Recommendation

Port the `_idleBalance` pattern from V1: only count Byzantine position growth as yield, never the live idle balance.

Remediation

The issue is now resolved by the patched code version validated by the commit **d2abeda.**

9

F-01 — Cooldown griefing via `_update`

- **Severity:** 🟢 Low — Score: **15** — Priority: **P1**
- **Category:** Griefing / transfer logic
- **Location:** `_update()` hook
- **Confidence:** High
- **Source:** Internal audit — identical finding in V1, not ported
- **Scoring note:** Initially classified Medium/40 in the preliminary analysis. Recalibrated to Low/15 after re-examination: a 4h maximum, self-resolving cooldown with no fund loss does not correspond to Availability=High. Remains **P1** because the fix is trivial and exploitation is free.

Description

The `_update` hook propagates the most recent timestamp during share transfers:

```
if (depositTimestamp[from] > depositTimestamp[to]) {
    depositTimestamp[to] = depositTimestamp[from];
}
```

An attacker sends 1 wei of share from a freshly deposited address to **forcibly extend a victim's cooldown**. Identical finding to V1, not carried over to V2.

Recommendation

Block the *outgoing* transfer during the cooldown rather than propagating the timestamp to the recipient.

Remediation

The issue is now resolved by the patched code version validated by the commit **d2abeda.**

F-05 — Treasury feeShares dilution

- **Severity:** 🟢 Low — Score: **37.6** — Priority: **P2**
- **Category:** Accounting logic / share minting
- **Location:** `harvest()` — `feeShares` calculation
- **Confidence:** High
- **Source:** Internal audit

Description

The fee shares mint depends on a fragile `denom`:

```
uint256 denom = current - toTreasury;
if (denom == 0) denom = 1;
uint256 feeShares = toTreasury
    * (totalSupply() + 10 ** _decimalsOffset()) / denom;
_mint(TREASURY, feeShares);
```

If `toTreasury` approaches `current` (yield spike or massive donation coupled with F-03), `denom` becomes tiny, minting a disproportionate number of shares to the treasury — massively diluting existing LPs.

Additional watch point: verify behavior at the **first harvest**, when `totalSupply()` is close to virtual shares only — the interaction with `_decimalsOffset() = 3` warrants a dedicated non-regression test.

Recommendation

To address the issue, implement a check to ensure that the number of fee shares minted does not exceed a predefined maximum threshold relative to the total supply, and trigger an error if this threshold is surpassed.

Remediation

The issue is now resolved by the patched code version validated by the commit **10fab7a**.

F-06 — `depositUpToCap()` reverts instead of returning `(0,0)`

- **Severity:** 🟢 Low — Score: **15** — Priority: **P3**
- **Category:** Business logic
- **Location:** `depositUpToCap()`
- **Confidence:** High
- **Source:** Internal audit

Description

When `remaining < minDeposit`, the function reverts instead of returning gracefully `(0, 0)`. Integrators calling `depositUpToCap` expect a non-reverting function by design — an unexpected revert can break automated workflows.

Recommendation

To prevent unexpected reverts, modify the function to return `(0, 0)` when the accepted amount is less than the minimum deposit, ensuring smoother integration and avoiding disruptions in automated workflows.

11

Remediation

The issue is now resolved by the patched code version validated by the commit **dd59bf2**.

F-07 — Missing `rescueToken()`

- **Severity: i** Informational — Priority: **P3**
- **Category:** Operational robustness
- **Source:** Internal audit

Description

No function allows recovery of ERC20 tokens accidentally sent to the contract. This is not an exploitable vulnerability, but a missing operational convenience.

Recommendation

Add a `rescueToken` function explicitly excluding `asset()` and Byzantine shares to prevent any misuse:

```
    function rescueToken(address token, address to, uint256 amount)
        external onlyOwner
    {
        if (token == asset()) revert CannotRescueAsset();
        if (token == address(BYZANTINE_VAULT)) revert
CannotRescueByzantineShares();
        IERC20(token).safeTransfer(to, amount);
    }
```

Remediation

The issue is now resolved by the patched code version validated by the commit **a9e2a22**.

F-08 — Incomplete resumption post-emergency exit

- **Severity: i** Informational — Score: **10** — Priority: **P3**
- **Category:** Operational robustness
- **Location:** `emergencyExitByzantine()` (or equivalent)
- **Source:** Internal audit

Description

After an emergency exit, funds remain as idle EURC in the vault. No dedicated function exists to automatically redeploy to Byzantine once the situation normalizes — the owner must orchestrate the resumption manually.

Recommendation

Add a `redeployToByzantine()` function restricted to the owner, callable only when unpaused, which redeploys the idle balance to Byzantine and updates `lastTotalAssets`.

Remediation

The issue is now resolved by the patched code version validated by the commit **f592ebd**.

E. Deployment Decision

All identified issues have been addressed, enabling the Paktol Organization to proceed with validating the deployment of the latest audited version of the code based on this commit 545cd06 at this stage of development. If any new modifications are made, we recommend conducting an additional audit to ensure the system's security remains intact. Additionally, appointing another auditor to perform a new security audit could enhance the diversity of methods and processes used.

Deployment Condition	Finding Remediation State
F-04 addressed	🟢 **Fixed**
F-02 addressed	🟢 **Fixed**
F-03 addressed	🟢 **Fixed**
F-05 addressed	🟢 **Fixed**
F-01 addressed	🟢 **Fixed**
F-06 addressed	🟢 **Fixed**
F-08, F-07 addressed	🟢 **Fixed**
Final score post-P0+P1+P2+P3	✅ **100 / 100**

4. AUDITING METHODOLOGIES

The smart contract audit was conducted using two primary methods—manual review and automated testing. Each method played a critical role in ensuring the quality, security, and reliability of the smart contract, with a focus on identifying vulnerabilities, optimizing performance, and verifying adherence to best practices and intended functionality.

A. Manual Review

Manual review is a crucial component of the audit process, as it allows auditors to identify issues that automated tools may overlook. This method involves a detailed, line-by-line examination of the smart contract code to ensure its correctness and compliance with the intended behavior. Key activities during the manual review included:

1. **Logic Validation:**
   - Ensuring that the implemented logic aligns with the specifications and expected behavior outlined in the documentation.
   - Identifying logical errors, inconsistencies, or potential edge cases that could lead to unexpected behavior.
2. **Economic Analysis:**
   - Assessing the economic design of the smart contract, including token distribution mechanisms and calculations.
   - Evaluating the contract's susceptibility to miner-extractable value (MEV) exploits or other economic vulnerabilities.
3. **Vulnerability Identification:**
   - Detecting security risks such as re-entrancy attacks, race conditions, and other vulnerabilities that could compromise the contract's integrity.
   - Reviewing access control mechanisms to ensure proper role-based permissions and security.
4. **Documentation Review:**
   - Verifying that code comments and documentation accurately describe the logic and functionality of the contract.
   - Ensuring that the documentation provides sufficient information for future developers to understand and maintain the contract.

B. Automated Review

Automated testing was employed to enhance the coverage and efficiency of the audit process. It included three main techniques: static analysis, dynamic analysis, and fuzzing.

1. **Static Analysis:** Static analysis involves the use of automated tools to examine the smart contract code without executing it. This phase focuses on detecting vulnerabilities and ensuring compliance with coding standards. Key activities included:

   - **Syntax and Style Checks:** Tools like Solhint were used to enforce coding standards and identify syntax errors.
   - **Security Vulnerability Detection:** Tools like Slither and Mythril were used to identify common vulnerabilities, such as integer overflows, underflows, and uninitialized storage variables.

- **Optimization Analysis:** Reviewing the code for redundant operations and gas inefficiencies.

2. **Dynamic Analysis :** Dynamic analysis involves executing the smart contract code in a controlled environment to observe its behavior during runtime. Key activities included:

   - **Simulation of Real-World Scenarios:** Using tools like Foundry to simulate interactions with the contract to detect runtime errors or vulnerabilities.
   - **Gas Usage Profiling:** Monitoring gas consumption during execution to identify inefficiencies and optimize performance.
   - **Stress Testing:** Evaluating the contract's behavior under high transaction loads to ensure scalability and reliability.

3. **Fuzzing :** Fuzzing is a technique used to test the robustness of the smart contract by providing it with unexpected or random inputs. Key activities included:

   - **Input Generation:** Using tools like Echidna to generate random inputs and test the contract's response to edge cases.
   - **Invariant Testing:** Ensuring that critical properties of the contract hold true under all conditions, such as proper token distribution and adherence to business rules.
   - **Error Detection:** Identifying potential vulnerabilities or crashes caused by unexpected inputs.

C. Analysis and Tools

Tool	Method	Version
Slither	Static analysis	0.11.5
Solhint	Static analysis	6.2.4
Mythril	Symbolic analysis	v0.24.8
Echidna	Fuzzing / invariants	2.3.3
Foundry	Dynamic analysis	1.7.1-stable
Manual review	Manual analysis (Economic / MEV)	—

5. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Audit Offering ("Agreement") duly signed and accepted by the client ("Customer" or the "Company").

This is a restricted audit report on our findings based on our analysis, in accordance with good industry practice as at the date of this report, in relation to: (i) smart contract best coding practices and issues in the framework and algorithms based on white paper, code, the details of which are set out in this report, (Smart Contract audit). To get a full view of audit analysis, it is crucial to read the

full report. While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely only on this report and cannot claim against us based on what it says or does not say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

This report provided in connection with the services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report could be publicly disclosed, referred to or relied upon by any person for any purposes, with the prior written consent from Crypto4All which will also deliver an audit certificate. This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Crypto4All to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology. Blockchain technology and cryptographic assets present a high level of ongoing risk. Crypto4All’s position is that each company and individual are responsible for their own due diligence and continuous security. Crypto4All goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze. Moreover, under any claim of any nature, whatsoever in any jurisdiction, in any way arising from or connected with this report and the use, inability to use or the results of use of this report, and any reliance on this report. The analysis of the Smart contract is purely based on the smart contract code shared with us alone.

The assessment services provided by Crypto4All is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

This report is provided for information purposes only and on a non-reliance basis and does not constitute investment advice. No one shall have any right to rely on the report or its contents, and Crypto4All and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers, and other representatives). Crypto4all make any warranty or representation to any person on the accuracy or completeness of the report. The report is provided "as is", without any conditions, warranties or other terms of any kind except as set out in this disclaimer, and Crypto4all hereby excludes all representations, warranties, conditions and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of

reasonable care and skill) which, but for this clause, might have effect in relation to the report. The analysis of the Smart contract is purely based on the smart contract code shared with us alone. This report and the associated services are governed by the laws of France. Any claims or disputes arising out of or in connection with this report shall be subject to the exclusive jurisdiction of the courts of France.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANYPRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED “AS IS” AND “AS AVAILABLE” AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND TO THE MAXIMUM EXTENT PERMITTED UNDERAPPLICABLE LAW, CRYPTO4ALL HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CRYPTO4ALL SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CRYPTO4ALL MAKES NOWARRANTY OF


C4 CRYPTO4ALL
BLOCKCHAIN ENGINEERING & CONSULTING